

Compte-rendu hebdomadaire

Travaux d'Étude et de Recherche

Semaine 5 — Du 27 avril au 1^{er} mai 2026

SUJET Recommandation

mangas & jeux vidéo

ENCADRANT JACQUENET François

ÉTUDIANT ALI ASSOUMANE Mtara

DATE 1^{er} mai 2026

Table des matières

1	Résumé de la semaine	2
2	Travail réalisé	2
2.1	Application Android — Écrans d'authentification	2
2.1.1	Structure du projet	2
2.1.2	Écrans implémentés	2
2.2	Application Android — Navigation entre écrans	2
2.3	Connexion Spring Boot ↔ Python	3
2.4	Prise en main de TFRS — Two-Tower Model	3
2.4.1	Principe du modèle	3
2.4.2	État de l'implémentation	4
3	Difficultés rencontrées	4
4	État d'avancement	4
5	Objectifs de la semaine prochaine	7
6	Conclusion	7

1 Résumé de la semaine

Cette cinquième semaine a été consacrée au développement de l'application Android, l'établissement de la connexion entre le backend Spring Boot et le moteur Python, et la prise en main de TensorFlow Recommenders (TFRS) pour le modèle *two-tower*.

2 Travail réalisé

2.1 Application Android — Écrans d'authentification

Le développement de l'application mobile Android a démarré cette semaine. Le projet a été créé sous Android Studio avec Jetpack Compose, l'approche moderne recommandée par Google pour construire des interfaces Android.

2.1.1 Structure du projet

Le projet a été organisé dès le départ en packages distincts pour séparer clairement les responsabilités :

- `data.api` : client Retrofit et interface `ApiService` pour les appels HTTP vers le backend Spring Boot
- `model` : classes de données `LoginRequest`, `RegisterRequest` et `AuthResponse`
- `ui.auth` : écrans Compose (`LoginScreen`, `RegisterScreen`) et `AuthViewModel`
- `navigation` : graphe de navigation `AppNavGraph`

2.1.2 Écrans implémentés

Deux écrans ont été développés en Jetpack Compose :

- **Écran de connexion** : champs nom d'utilisateur et mot de passe, bouton de connexion avec indicateur de chargement, affichage des erreurs réseau, lien vers l'inscription
- **Écran d'inscription** : champs nom d'utilisateur, email et mot de passe, même gestion des états et des erreurs

La logique de chaque écran est gérée dans un `AuthViewModel` séparé, ce qui permet de distinguer clairement ce qui relève de l'interface et ce qui relève du traitement des données.

2.2 Application Android — Navigation entre écrans

La navigation entre les écrans a été mise en place à l'aide de la bibliothèque *Navigation Compose*. Un graphe de navigation centralisé (`AppNavGraph`) définit trois destinations :

- `login` : point d'entrée par défaut de l'application
- `register` : accessible depuis l'écran de connexion

- `home` : destination après authentification réussie (écran provisoire en attente du catalogue)

La navigation utilise `popUpTo` pour vider la pile arrière lors d'une connexion réussie, empêchant l'utilisateur de revenir aux écrans d'authentification via le bouton retour.

2.3 Connexion Spring Boot ↔ Python

La communication entre le backend Spring Boot et le moteur de recommandation Python Flask a été établie cette semaine. Trois composants ont été ajoutés au projet Spring Boot :

- `AppConfig.java` : configuration du bean `RestTemplate` utilisé pour les appels HTTP sortants
- `PythonEngineService.java` : service encapsulant les appels vers Flask — méthode `isHealthy()` pour vérifier la disponibilité du moteur, et `getRecommendations()` pour déclencher le calcul
- `RecommendationController.java` : enrichi de deux nouveaux endpoints — `GET /api/recommendations/health` (vérifie que Flask est joignable) et `GET /api/recommendations/` (appel direct au moteur Python)

L'URL du moteur Python est externalisée dans `application.properties`, ce qui facilite la reconfiguration selon l'environnement. Le test de validation via Thunder Client confirme que Spring Boot contacte bien Flask et obtient une réponse 200 OK :

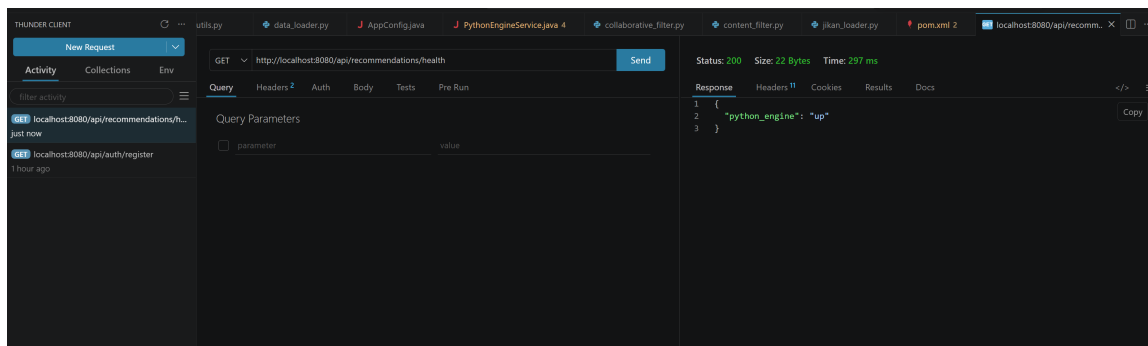


FIGURE 1 – Validation de la connexion Spring Boot ↔ Python — réponse 200 OK

2.4 Prise en main de TFRS — Two-Tower Model

TensorFlow Recommenders (TFRS) a été installé et configuré dans l'environnement virtuel Python. L'architecture *two-tower* a été implémentée dans un fichier dédié `two_tower_model.py`.

2.4.1 Principe du modèle

L'idée du modèle two-tower est simple : on apprend à représenter chaque utilisateur et chaque item (manga ou jeu) sous forme d'un vecteur numérique. Plus deux vecteurs sont proches, plus l'item correspond aux goûts de l'utilisateur.

Le modèle est composé de deux parties :

- **User Tower** : prend en entrée l'identifiant d'un utilisateur et produit un vecteur de 32 valeurs qui résume ses préférences
- **Item Tower** : fait la même chose pour chaque item du catalogue

Pendant l'entraînement, le modèle apprend à rapprocher les vecteurs des utilisateurs de ceux des items qu'ils ont appréciés. Une fois entraîné, recommander des items à un utilisateur revient à chercher les items dont le vecteur est le plus proche du sien.

2.4.2 État de l'implémentation

Le fichier est fonctionnel et prêt pour l'entraînement. En l'absence de données de notation utilisateur en base, le moteur détecte automatiquement le cas *cold start* et retourne un avertissement :

WARNING - Aucun rating en base - Two-Tower ignoré

L'entraînement sur données synthétiques est prévu pour la semaine 6.

3 Difficultés rencontrées

- **Incompatibilité JDK / Lombok** : le JDK installé (version 25) est trop récent pour Lombok, entraînant des échecs de compilation via Maven. La résolution a nécessité l'installation d'un JDK 21 LTS et la reconfiguration du `pom.xml` avec le plugin `maven-compiler-plugin` et `annotationProcessorPaths`.
- **Incompatibilité TFRS / Keras 3** : TFRS requiert Keras 2, alors que TensorFlow 2.21 installe Keras 3 par défaut. Résolu en définissant la variable d'environnement `TF_USE_LEGACY_KERAS=1` avant l'import.
- **Sécurité Spring Boot** : l'endpoint `/api/recommendations/health` retournait une erreur 403 car non déclaré comme public dans `SecurityConfig`. Résolu en ajoutant un `requestMatchers().permitAll()`.

4 État d'avancement

Tâche	Description	Statut
<i>Architecture & Base de données</i>		
Architecture technique	Schéma BDD, diagramme global	Terminé
Base de données	Création MySQL, 7 tables, relations	Terminé

Tâche	Description	Statut
Alimentation mangas	Jikan API — 498 mangas, 77 genres	Terminé
Alimentation jeux vidéo	IGDB API — catalogue jeux	Reporté (S6)
<i>Backend Spring Boot</i>		
Entités JPA & repositories	Mapping objet-relationnel	Terminé
Controllers REST	16 endpoints, 6 controllers	Terminé
Sécurité JWT	JwtUtil, JwtFilter, Security-Config	Terminé
Tests endpoints	Validation via Thunder Client	Terminé
Connexion Spring Boot ↔ Python	Appels REST inter-services	Terminé
Gestion erreurs & pagination	Codes HTTP, résultats paginés	À faire (S8)
<i>Moteur de recommandation Python</i>		
SVD tronquée (numpy)	Filtrage collaboratif matriciel	Terminé
KNN collaboratif	Recherche des utilisateurs similaires	Terminé
TF-IDF + hybride	Filtrage contenu, cold start	Terminé
API Flask	Endpoints /health, /recommend	Terminé
Two-tower model (TFRS)	Architecture implémentée, prêt à entraîner	En cours (S6)

Tâche	Description	Statut
Entraînement TFRS	Données synthétiques, évaluation	À faire (S6)
Tests & optimisation	Validation, mise en cache	À faire (S6)
<i>Application Android (Kotlin)</i>		
Apprentissage Kotlin	Syntaxe, classes, null safety	Terminé
Écran connexion / inscription	Login, inscription, validation	Terminé
Navigation entre écrans	NavGraph, destinations, back stack	Terminé
Intégration Retrofit + JWT	Appels HTTP sécurisés	En cours (S6)
Catalogue mangas & jeux	Liste, filtres genre / type	À faire (S6)
Fiche détail item	Synopsis, note, genres	À faire (S7)
Système de notation in-app	Notes utilisateur 1–10	À faire (S7)
Écran recommandations	Recommandations personnalisées	À faire (S8)
Wishlist	Ajout / suppression d'items	À faire (S8)
Tests end-to-end	Scénarios complets mobile → reco	À faire (S9)
<i>Expérimentations & livraisons</i>		
Expérimentations	Évaluation des recommandations générées	À faire (S9–S10)

Tâche	Description	Statut
Comparaison SVD vs TFRS	Évaluation comparative	À faire (S10)
Rapport final	Rédaction continue (S3–S12)	En cours
Relecture & mise en page	Figures, diagrammes, corrections	À faire (S11)
Démo application	Démonstration fonctionnelle complète	À faire (S11–S12)

5 Objectifs de la semaine prochaine

- Entraîner le modèle two-tower sur des données synthétiques et évaluer ses performances (précision, rappel)
- Développer l'écran catalogue mangas dans l'application Android avec appels Retrofit vers Spring Boot
- Intégrer le JWT dans les requêtes Retrofit (header `Authorization: Bearer <token>`)
- Alimenter la base de données avec le catalogue jeux vidéo via l'API IGDB

6 Conclusion

Cette cinquième semaine a été productive : les trois parties du projet avancent en parallèle et commencent à se connecter entre elles. L'application Android affiche ses premiers écrans, le backend Spring Boot peut désormais appeler le moteur Python, et le modèle two-tower est prêt à être entraîné. La semaine prochaine, l'objectif est de lancer le premier entraînement réel du modèle TFRS et de continuer le développement de l'application mobile.